

Módulo 10 – Capítulo 02 – Sección 01

Arquitectura de referencia para una plataforma de IA empresarial

Una arquitectura de referencia responde a una pregunta fundamental: ¿cómo se ensamblan los componentes de la plataforma en un sistema coherente? El Capítulo 01 describió qué hace cada componente por separado; este capítulo responde cómo se conectan entre sí con contratos de API versionados, qué patrón arquitectónico organiza esas conexiones, y cómo esa estructura se convierte en un sistema que múltiples equipos pueden usar simultáneamente sin interferirse. La arquitectura de referencia no es un diagrama aspiracional: es un conjunto de decisiones implementadas, testeadas y documentadas que los equipos pueden adoptar sin tener que redescubrir los mismos problemas de infraestructura.

El patrón más adoptado en la industria organiza la plataforma en cinco **planos funcionales**, cada uno con responsabilidades bien delimitadas y contratos de API explícitos con los planos adyacentes. Este patrón, documentado por empresas como Lyft, Netflix y Meta en sus engineering blogs, tiene la ventaja de que cada plano puede evolucionar de forma independiente sin romper los contratos con los demás, y cada equipo puede ser responsable de uno o varios planos sin necesitar conocimiento completo del sistema.

El **plano de datos** es la base sobre la que opera todo lo demás: ingesta datos de múltiples fuentes (Kafka o Pub/Sub para streaming, Spark o dbt para procesamiento batch), los almacena en formato columnar versionado (Delta Lake o Apache Iceberg sobre S3) que soporta time-travel queries para reproducibilidad de experimentos, y expone features procesadas a través del feature store (Feast o Tecton) con consistencia garantizada entre el almacén offline usado en entrenamiento y el almacén online usado en inferencia. El plano de datos es también donde se materializan los controles de governance sobre los datos: clasificación de sensibilidad, políticas de acceso y retención, y validaciones de calidad que impiden que datos incorrectos propaguen sus errores al resto del sistema.

El **plano de experimentación** orquesta el ciclo de desarrollo de modelos: el cluster de Kubernetes con GPU pools segmentados por prioridad (interactivo para notebooks, batch

para training jobs de larga duración), el sistema de tracking de experimentos (MLflow o Weights & Biases) que registra automáticamente hiperparámetros, métricas y artefactos de cada run, y las herramientas de hyperparameter optimization (Optuna o Ray Tune) que automatizan la búsqueda del espacio de configuraciones. Este plano debe ser accesible para los ML Engineers con el menor overhead posible: lanzar un experimento no debería requerir más que especificar el código, los datos y los recursos necesarios.

El **plano de serving** transforma modelos registrados en endpoints de producción: el model server (Triton Inference Server o KServe) gestiona la carga del modelo y el procesamiento de requests, el API Gateway (Kong, AWS API Gateway) aplica autenticación, rate limiting y logging al tráfico entrante, y el sistema de routing (Istio o un componente custom) dirige el tráfico entre versiones del modelo para estrategias de canary, blue-green o A/B testing. Cada equipo consumidor del endpoint percibe un servicio estable y con SLA definido; la complejidad del routing y la gestión de versiones es transparente para ellos.

El **plano de control** es el sistema nervioso de la plataforma: el model registry con webhooks de promoción automática, el orquestador de pipelines con DAGs declarativos (Kubeflow Pipelines o Prefect), la gestión de configuración via GitOps (ArgoCD), y la gestión de secretos (HashiCorp Vault o AWS Secrets Manager). El plano de control es lo que hace que la plataforma sea autónoma: un modelo que pasa los gates de calidad en el registry puede desplegarse automáticamente sin intervención humana porque el plano de control ejecuta la secuencia correcta de operaciones.

El **plano de observabilidad** cierra el ciclo con métricas de sistema en Prometheus, métricas de calidad del modelo en Evidently AI o WhyLabs con drift detection automático, logs estructurados en Elasticsearch para debugging, y traces distribuidos en Jaeger u OpenTelemetry para trazabilidad de requests complejos. Los equipos acceden a dashboards predefinidos en Grafana sin configurar el sistema de métricas manualmente: la observabilidad es el comportamiento por defecto de cualquier servicio que corre en la plataforma.

Componentes principales de la arquitectura de referencia

- **Data plane:** ingesta streaming (Kafka) y batch (Spark en EMR o Dataproc), almacenamiento en object store con formato columnar versionado (Delta Lake), feature store con consistencia online/offline garantizada.

- **Training plane:** Kubernetes con GPU pools segmentados por prioridad, job scheduling con Volcano o Kueue, distributed training con PyTorch DDP y checkpointing en S3 para tolerancia a fallos.
- **Serving plane:** model server (Triton/KServe) con autoscaling via KEDA, API Gateway con rate limiting y autenticación JWT, routing por canary weight o header-based routing.
- **Control plane:** model registry con webhooks de promoción, pipeline orchestrator con DAGs declarativos, GitOps con ArgoCD, secret management con HashiCorp Vault.
- **Observability plane:** métricas de sistema en Prometheus, métricas de modelo con drift detection, logs estructurados en Elasticsearch, traces distribuidos en Jaeger.

Nota del Arquitecto: La arquitectura de referencia se documenta en el developer portal (Backstage) de la organización, incluyendo los contratos de API entre planos y los criterios de decisión para cuándo usar cada componente. Sin esa documentación, cada equipo nuevo que llega a la plataforma reconstruye el mapa del sistema hablando con personas, en lugar de consultando una fuente de verdad. El costo de mantener esa documentación actualizada es insignificante comparado con el costo de onboarding que se paga cuando no existe.

Una arquitectura de referencia no es un diagrama: es un conjunto de decisiones implementadas, testeadas y documentadas que los equipos pueden adoptar sin tener que redescubrir los mismos problemas de infraestructura. La siguiente sección examina cómo el self-service sobre esta arquitectura elimina la necesidad de que el equipo de plataforma actúe como intermediario en cada despliegue, liberando su capacidad para mejorar la infraestructura en lugar de operar como helpdesk.

Módulo 10 – Capítulo 02 – Sección 02

Self-service: diseñar para que los equipos puedan desplegar sin intervención del equipo de plataforma

El principio de self-service en una plataforma de IA establece que un equipo de AI Engineering debe poder pasar de un modelo entrenado a un endpoint de producción sin abrir un ticket, sin esperar aprobación manual de infraestructura, y sin necesitar conocimiento profundo de Kubernetes o cloud networking. Esta afirmación suena razonable en abstracto, pero su implementación técnica requiere resolver un problema de diseño no trivial: ¿cómo se garantiza que los equipos desplieguen de forma segura y conforme a las políticas de la organización cuando no hay nadie de plataforma en el loop? La respuesta es desplazar los controles del proceso manual al proceso automatizado: los guardrails no se eliminan, se automatizan.

Los "golden paths" son el vehículo principal del self-service: flujos de trabajo predefinidos, probados y automatizados que cubren el 80% de los casos de uso más comunes, expuestos como comandos CLI o como templates de pipeline que el equipo completa con sus parámetros específicos. Un ejemplo concreto: el comando `platform deploy --model my_model:v2.1 --env prod` no es una interfaz simple sobre `kubectl apply` —es una secuencia automática de validaciones de seguridad de la imagen (Trivy), tests de carga (k6), verificación de que el modelo pasa sus eval gates, y actualización del config repo de GitOps para que ArgoCD aplique el cambio. El equipo de AI Engineering ve un comando; la plataforma ejecuta una secuencia de quince operaciones garantizadas. En plataformas como Lyft y Spotify, este patrón redujo el tiempo de despliegue de semanas a horas, con el equipo de plataforma respondiendo a incidentes de infraestructura en lugar de actuar como intermediario en cada despliegue.

La automatización de self-service requiere **guardrails técnicos integrados** en el propio flujo de despliegue. Las validaciones de seguridad de imágenes —con Trivy o Snyk escaneando la imagen Docker antes de permitir el push al registry— son un ejemplo: no se puede deployar una imagen con vulnerabilidades críticas conocidas, no porque haya alguien revisando, sino porque el pipeline rechaza la operación antes de que llegue a producción. Los tests de carga

automáticos antes de promover a producción —con Locust o k6 ejecutando contra el endpoint en staging— garantizan que el SLO de latencia se cumple antes de que el endpoint reciba tráfico real. Los gates de calidad de modelo que comparan métricas offline del modelo candidato contra el modelo actualmente en producción previenen regresiones silenciosas. La clave es que todos estos controles son técnicos e inevitables: no se pueden bypassear sin romper el flujo de despliegue.

La experiencia del desarrollador en el self-service es tan importante como los controles técnicos. Un CLI con mensajes de error crípticos convierte el self-service en una fuente de frustración: el equipo abre un ticket al equipo de plataforma de todas formas, pero ahora también tiene que explicar el mensaje de error que no entendió. El diseño de CLIs efectivos para plataformas de IA requiere los mismos principios que el diseño de APIs públicas: mensajes de error que explican qué falló y cómo corregirlo, comandos de diagnóstico que muestran el estado del sistema, y documentación con ejemplos ejecutables que funcionan en la primera prueba. La diferencia entre "error: 403 Forbidden" y "error: el equipo nlp-squad no tiene permisos para desplegar en el namespace production-ml; solicita acceso en #platform-access-requests" es la diferencia entre una llamada de soporte y un problema resuelto en 30 segundos.

Aspectos técnicos del self-service

- **CLI interno:** Python con Click o Typer que encapsula kubectl, helm, docker build y llamadas al model registry en un comando único con flags explícitos y mensajes de error accionables que describen la causa raíz y la solución sugerida.
- **Templates de pipeline:** KubeFlow Pipelines YAML o Prefect flows con parámetros de input/output estandarizados y steps obligatorios de validación no bypassables; los equipos personalizan los parámetros, no la estructura de control.
- **Portales de self-service:** Backstage con Software Templates que generan scaffolding de proyectos con CI/CD preconfigurado y acceso al cluster de training; el nuevo equipo ejecuta un template y obtiene un repositorio funcional en minutos.
- **Policy as Code:** Open Policy Agent (OPA) o Kyverno validan cada recurso de Kubernetes en el momento de su creación, rechazando configuraciones que violen las políticas de seguridad o de cuotas antes de que lleguen al cluster de producción.
- **Runbooks automatizados:** playbooks de remediación para los problemas más comunes (pod con OOMKilled, feature store sync retrasado, endpoint con error rate elevado) que los equipos pueden ejecutar de forma autónoma sin escalar al equipo de plataforma.

Nota del Arquitecto: *El self-service no significa ausencia de control: significa que los controles están integrados automáticamente en el camino de despliegue, de modo que cumplir las políticas es el camino de menor resistencia. Cuando un ingeniero intenta bypassar el proceso de self-service —por ejemplo, aplicando kubectl directamente en lugar de usar el CLI— el sistema debe detectarlo y notificar, pero no por razones punitivas: porque ese deploy bypass no tiene los controles de seguridad ni el registro de auditoría que el proceso automatizado garantiza.*

El self-service efectivo libera al equipo de plataforma de ser el cuello de botella de cada despliegue y lo posiciona en su rol correcto: mejorar la infraestructura, responder a incidentes, y evolucionar los golden paths para cubrir nuevos casos de uso. La siguiente sección aborda cómo el multi-tenancy garantiza que el self-service de un equipo no afecte a los demás equipos que comparten la misma infraestructura.

Módulo 10 – Capítulo 02 – Sección 03

Multi-tenancy: aislamiento de recursos, costos y datos entre equipos

El multi-tenancy en una plataforma de IA resuelve un problema de coordinación fundamental: múltiples equipos comparten la misma infraestructura física, pero cada uno necesita la garantía de que sus operaciones no afectarán a los demás. Sin aislamiento efectivo, emergen tres categorías de problemas que destruyen la confianza en la plataforma. El "noisy neighbor" ocurre cuando el training job de un equipo consume toda la memoria del nodo compartido y degrada la latencia de los endpoints de producción de otro. El acceso cruzado de datos ocurre cuando un bug de configuración permite que un equipo lea los datasets de modelos propietarios de otro. La contaminación de costos ocurre cuando los recursos de un proyecto aparecen facturados en el presupuesto de otro porque el tagging de recursos era incorrecto o inconsistente. El multi-tenancy efectivo previene los tres simultáneamente.

En Kubernetes, el aislamiento de cómputo se implementa mediante **namespaces** como unidad de separación entre equipos, con **ResourceQuotas** que establecen límites máximos de CPU, memoria y GPU por namespace: un equipo no puede consumir más recursos de los que tiene asignados por política, independientemente de cuánto exija el job. Las **NetworkPolicies** restringen el tráfico entre namespaces: por defecto, ningún servicio de un equipo puede comunicarse con los servicios de otro equipo a menos que exista una política explícita de egress/ingress que lo permita. Las **PriorityClasses** garantizan que los workloads de producción (alta prioridad) no sean desalojados por workloads de experimentación (baja prioridad) cuando el cluster está bajo presión de recursos: un job de training puede ser interrumpido para liberar recursos para un endpoint de producción, pero nunca al revés.

El aislamiento de datos requiere una estrategia diferente para el object storage y para el feature store. En S3 o GCS, cada equipo recibe prefijos de bucket propios con políticas de IAM que previenen acceso cruzado: las condiciones de IAM que incluyen `aws:ResourceTag/team` como condición de la política garantizan que solo el equipo propietario puede leer y escribir en sus rutas de almacenamiento. Para datos de alta sensibilidad, el aislamiento puede

reforzarse con KMS keys distintas por equipo: incluso si alguien obtuviera acceso no autorizado a los bytes cifrados, la clave de descifrado pertenece a un KMS key ID diferente. En el feature store, Feast implementa el aislamiento mediante proyectos separados con entity ownership por equipo: las features de un equipo no son visibles en el namespace de otro equipo a menos que se exporten explícitamente como features compartidas con un acuerdo de SLA.

La atribución de costos en un entorno multi-tenant requiere etiquetado obligatorio y automatizado de recursos. La convención de tags — `team=nlp-squad`, `project=search-ranking`, `environment=production`, `cost_center=engineering` — debe ser validada en el momento de creación del recurso por un webhook de admisión de Kubernetes que rechaza pods sin las etiquetas requeridas: si el tag no está presente, el pod no se crea. Este enforcement técnico garantiza que el 100% de los recursos estén correctamente atribuidos, no el 95%: el 5% sin atribuir en un cluster de GPU puede representar miles de dólares de gasto invisible. Los tags se propagan automáticamente a AWS Cost Explorer o GCP Billing, donde herramientas como Kubecost calculan el costo exacto por namespace, deployment y pod y generan reportes semanales de gasto por equipo.

Puntos críticos del multi-tenancy

- **Aislamiento de cómputo:** Kubernetes LimitRanges y ResourceQuotas por namespace, con PriorityClasses para garantizar que los jobs de producción no sean desalojados por jobs de experimentación; el scheduler respeta la jerarquía automáticamente.
- **Aislamiento de red:** NetworkPolicies que permiten solo el tráfico explícitamente autorizado; los servicios de plataforma (registry, feature store) tienen egress permitido desde todos los namespaces bajo autenticación, sin exponer su implementación interna.
- **Aislamiento de datos:** object storage con prefijos por equipo y IAM conditions que impiden acceso cruzado; encriptación por equipo con KMS keys distintas para datos clasificados como confidenciales o sensibles.
- **Chargeback técnico:** etiquetado obligatorio de recursos validado por webhook de admisión; reportes automáticos de costo por equipo generados semanalmente con granularidad de proyecto y environment.
- **Aislamiento en el registry:** MLflow usa experiments con permisos por usuario/equipo; SageMaker Model Registry usa model groups con política de IAM que restringe acceso por equipo; ningún modelo puede ser visto por equipos no autorizados.

Nota del Arquitecto: *El multi-tenancy exitoso hace invisible la compartición de recursos: cada equipo percibe que tiene su propia infraestructura, sin saber que está compartiendo el mismo cluster de A100s con otros diez equipos. El test de que el multi-tenancy funciona correctamente no es técnico sino conductual: cuando los equipos dejan de preguntar "¿mi experimento está afectando los endpoints de otro equipo?" y confían en que el sistema previene esa situación, el aislamiento ha sido exitoso.*

El multi-tenancy en una plataforma de IA hace que la compartición de recursos sea transparente e invisible para los equipos que los comparten. La siguiente sección examina las abstracciones de API que son el contrato concreto entre la plataforma y los equipos consumidores: la capa que hace que toda esa complejidad subyacente sea accesible sin necesidad de conocerla en detalle.

Módulo 10 – Capítulo 02 – Sección 04

Abstracciones de plataforma: APIs internas que ocultan complejidad de infraestructura

Las abstracciones de plataforma son los contratos de interfaz que definen cómo los equipos consumidores interactúan con la infraestructura sin necesitar conocer su implementación subyacente. Un equipo de AI Engineering debe poder lanzar un training job especificando solo el número de GPUs, la imagen Docker y el script de entrada —sin configurar manualmente nodos de Kubernetes, seleccionar instance types, o gestionar tolerations y affinities de scheduling. Esta separación entre lo que el usuario especifica y lo que la plataforma ejecuta es la esencia del diseño de abstracciones: el usuario trabaja en el vocabulario del problema de ML, la plataforma trabaja en el vocabulario de la infraestructura.

El diseño de abstracciones efectivas sigue el principio de **progressive disclosure**: la interfaz simple cubre el 80% de los casos de uso con el mínimo de parámetros necesarios, pero permite que los usuarios avanzados accedan a configuraciones más granulares via parámetros opcionales sin romper la API estable. Un ejemplo concreto: la Training Job API puede tener como interfaz mínima `platform.submit_training(script="train.py", gpus=4)` para los casos más simples, y como extensión opcional `platform.submit_training(script="train.py", gpus=4, distributed_strategy="ddp", checkpoint_every_n_steps=500, spot_instance=True)` para usuarios que necesitan control fino. La interfaz simple funciona para el 80% de los jobs; los parámetros adicionales cubren el 20% restante sin obligar a todos los usuarios a entender los detalles de la capa de distributed training.

Una abstracción incorrecta se reconoce por sus síntomas: los equipos escriben workarounds para eludir la abstracción, o la bypasen directamente usando los sistemas subyacentes. Si los ingenieros prefieren ejecutar `kubectl apply -f my-job.yaml` manualmente en lugar de usar el CLI de la plataforma, es porque la abstracción del CLI no les da lo que necesitan, o porque usarla es más complicado que configurar Kubernetes directamente. Ambas señales son feedback valioso: la abstracción tiene un gap de capacidad o un problema de usabilidad que el equipo de plataforma debe resolver. En términos de diseño de productos, una

abstracción bypassada habitualmente es un producto que los usuarios han abandonado sin declararlo formalmente.

Las abstracciones de plataforma se implementan típicamente en tres formatos complementarios. El **SDK Python** es el formato más conveniente para los ML Engineers que trabajan en notebooks o scripts: `wandb.init()` de Weights & Biases y `Estimator` de SageMaker son ejemplos de abstracciones SDK que redujeron la fricción de sus respectivos ecosistemas. El **CLI interno** es el formato preferido para operaciones que se integran en scripts de CI/CD y pipelines de automatización: comandos como `platform deploy`, `platform logs`, `platform status` dan visibilidad del sistema sin requerir programación. La **API REST con documentación OpenAPI** es el formato más flexible para integraciones con herramientas de CI/CD y sistemas de terceros que necesitan consumir la plataforma programáticamente. Los tres formatos deben exponer la misma funcionalidad subyacente con el mismo comportamiento: la consistencia entre formatos es lo que hace que el aprendizaje de uno facilite el uso de los demás.

Conceptos clave de las abstracciones de plataforma

- **Training Job API:** abstrae la complejidad de Kubernetes Job/PyTorchJob/TFJob en una interfaz simple `platform.submit_training(config)` que selecciona automáticamente el runtime correcto según el framework detectado y los recursos solicitados.
- **Serving API:** `platform.deploy_model(model_uri, resources)` que crea automáticamente el InferenceService de KServe, configura el ingress, registra el endpoint en el service catalog y habilita monitoring; el equipo no toca YAML de Kubernetes.
- **Data API:** acceso a datasets y features mediante rutas semánticas (`platform.load_dataset("user_events/v3")`) que resuelven automáticamente a la ubicación en S3 con las credenciales correctas para el equipo llamante.
- **Experiment API:** wrappers sobre MLflow o W&B que añaden automáticamente metadatos corporativos (team, project, cost_center) a cada run y enforcement de convenciones de naming, garantizando que todos los experiments sean buscables y atribuibles.
- **Versioning contract:** toda abstracción expuesta tiene una versión semántica explícita y un período de deprecación anunciado de al menos 90 días antes de su eliminación; los equipos consumidores pueden planificar sus migraciones sin sorpresas.

Nota del Arquitecto: La prueba más reveladora de una abstracción es pedirle a un ML Engineer que nunca la usó que complete una tarea común con ella —sin documentación, solo con el comando de ayuda— y medir cuánto tiempo tarda y cuántas veces se equivoca. Si el tiempo es mayor de 10 minutos o hay más de dos errores, la abstracción necesita rediseño. Las mejores abstracciones son las que se descubren intuitivamente: `platform deploy --model x --env prod` no requiere manual.

Una abstracción de plataforma exitosa es aquella que un ML Engineer adopta porque le facilita el trabajo, no porque se lo exijan. La siguiente sección examina el proceso de evolución de la plataforma: cómo se pasa de los scripts ad-hoc iniciales a una plataforma gobernada con SLOs y roadmap, y qué señales indican que es momento de invertir en el siguiente nivel de madurez.

Módulo 10 – Capítulo 02 – Sección 05

Evolución de la plataforma: de scripts ad-hoc a plataforma gobernada

Ninguna organización construye una plataforma de IA completa en un sprint. La realidad del crecimiento de plataformas en la industria sigue un patrón bien documentado de maduración progresiva, donde cada nivel resuelve los problemas del nivel anterior y crea las precondiciones para el siguiente. Comprender esta progresión es útil en dos sentidos: permite diagnosticar en qué nivel se encuentra la organización y qué inversiones son prioritarias en este momento, y evita el error de intentar construir el nivel 3 sin haber consolidado el nivel 2, lo que invariablemente resulta en complejidad prematura que nadie adopta.

El **Nivel 0** —scripts ad-hoc— es donde comienza toda organización que introduce ML en su stack técnico. Los data scientists entrenan modelos en notebooks de Jupyter con datos en su directorio local, los despliegan mediante SCP o rsync a un servidor compartido, y nadie fuera del equipo inmediato entiende qué modelo está en producción ni cómo fue creado. No hay versionado de experimentos, no hay reproducibilidad, y el "despliegue" es literalmente copiar archivos. Este nivel es aceptable durante las fases de exploración y proof of concept, pero se convierte rápidamente en un cuello de botella cuando el primer modelo de producción necesita ser actualizado y nadie recuerda exactamente cómo fue generado.

El **Nivel 1** —automatización básica— emerge cuando algunos ingenieros convierten sus scripts ad-hoc en código versionado con Git, añaden un CI/CD básico con GitHub Actions, y empiezan a usar MLflow para tracking de experimentos. Este nivel es frágil pero funcional: los experimentos son reproducibles si el ingeniero que los corrió sigue en la organización, los despliegues son más consistentes pero siguen siendo manuales, y el monitoreo de modelos en producción es casi inexistente. La señal de que es momento de invertir en el Nivel 2 es cuando los scripts del Nivel 1 se rompen en el entorno de otro ingeniero ("en mi máquina funciona"), cuando el equipo pasa más tiempo debuggeando el pipeline que mejorando el modelo, o cuando hay más de dos personas que necesitan acceso al mismo ambiente de entrenamiento y empiezan a pisarse entre sí.

El **Nivel 2** —plataforma naciente— requiere la primera inversión real en infraestructura compartida: un cluster de Kubernetes para scheduling de jobs, APIs internas con versiones que abstraen la infraestructura de los ML Engineers, un model registry compartido donde todos los experimentos se registran, un feature store inicial que garantiza consistencia entre entrenamiento e inferencia, y observabilidad básica con Prometheus. Este nivel ya exige un equipo dedicado (aunque sea parcialmente) a mantener la plataforma, porque la complejidad de los sistemas interconectados supera lo que puede gestionarse como proyecto paralelo de un ingeniero a tiempo parcial.

El **Nivel 3** —plataforma gobernada— es el estado objetivo para organizaciones con múltiples equipos de AI Engineering: un IDP completo con self-service, multi-tenancy con aislamiento garantizado, SLOs contractuales medidos y publicados, chargeback automatizado, y un equipo dedicado con roadmap público y feedback loop activo. En este nivel, la plataforma es indistinguible de un producto SaaS interno: tiene releases con notas de cambio, documentación mantenida en un developer portal, un proceso de soporte con SLAs de respuesta, y métricas de adopción publicadas trimestralmente.

Etapas de evolución técnica

- **Nivel 0 – Scripts ad-hoc:** entrenamiento manual con notebooks, despliegue manual vía SCP o rsync, sin tracking de experimentos ni versionado de modelos; aceptable para PoC, insostenible en producción.
- **Nivel 1 – Automatización básica:** scripts en Git, CI/CD básico con GitHub Actions, MLflow para tracking, Docker para reproducibilidad de entornos; frágil pero funcional para equipos pequeños.
- **Nivel 2 – Plataforma naciente:** Kubernetes para scheduling, APIs internas versionadas, model registry compartido, feature store inicial, observabilidad básica; requiere inversión y equipo parcialmente dedicado.
- **Nivel 3 – Plataforma gobernada:** IDP completo con self-service, multi-tenancy, SLOs contractuales, chargeback automatizado, equipo dedicado con roadmap público; el estándar para organizaciones maduras con múltiples equipos de IA.
- **Señales de regresión:** crecimiento del número de repos privados con herramientas de infraestructura duplicadas, aumento del tiempo medio de despliegue, incremento de incidentes causados por diferencias de configuración entre equipos.

Nota del Arquitecto: *El paso más crítico en la evolución de la plataforma no es el de Nivel 2 a Nivel 3, sino el de Nivel 1 a Nivel 2. En ese momento es necesario refactorizar los scripts ad-hoc en APIs con contratos explícitos, añadir tests automatizados, y crucialmente, asignar ownership claro a alguien responsable de mantener el sistema. Sin esa asignación explícita de responsabilidad, los sistemas del Nivel 2 se degradan gradualmente de vuelta al Nivel 1 porque nadie los mantiene activamente.*

La evolución de la plataforma debe ser incremental y guiada por el dolor real de los equipos, no por un roadmap aspiracional. La sección de cierre de este capítulo sintetiza el argumento central: una plataforma que funciona se percibe como un multiplicador de productividad, no como burocracia, y esa percepción se construye o se destruye en los detalles de la experiencia de uso diaria.

Módulo 10 – Capítulo 02 – Sección 06

Cierre: una buena plataforma se percibe como un multiplicador de productividad, no como burocracia

El test definitivo del diseño de una plataforma de IA es simple y brutalmente honesto: ¿los equipos la usan voluntariamente o la evitan siempre que pueden? Esta pregunta no admite respuestas ambiguas. Una plataforma que se percibe como burocracia tiene síntomas técnicos identificables: los ingenieros prefieren configurar su propio cluster de Kubernetes antes de aprender la API de la plataforma, crean instancias de EC2 directas para eludir el scheduler de jobs, o construyen sus propios wrappers de MLflow en lugar de usar el SDK interno. Cada una de estas decisiones es racional desde la perspectiva del ingeniero individual: el camino de menor resistencia pasa por fuera de la plataforma, no por dentro.

Cuando una plataforma funciona como multiplicador de productividad, los tiempos de ciclo se reducen de forma visible y medible. Un fine-tuning que antes requería tres días de configuración de infraestructura se lanza en 30 minutos. El proceso de despliegue que requería intervención del equipo de plataforma se completa de forma autónoma en menos de una hora. Los incidentes de producción relacionados con configuración incorrecta de infraestructura desaparecen de los postmortems porque la plataforma elimina las decisiones de configuración que los causaban. Los equipos no necesitan coordinar con el equipo de plataforma para operaciones rutinarias, liberando esa capacidad de coordinación para problemas genuinamente complejos que sí requieren colaboración.

El diseño para la productividad implica decisiones técnicas concretas que van más allá de la arquitectura de componentes. Los mensajes de error deben ser accionables: en lugar de "error 403 Forbidden", el sistema debe producir "error: el equipo nlp-squad no tiene permisos para desplegar en el namespace production-ml; solicita acceso enviando un mensaje en #platform-access-requests con el template `!request-access nlp-squad production-ml`". La documentación debe incluir ejemplos de código que funcionan en la primera ejecución, no ejemplos que asumen un estado del sistema que el lector debe conocer previamente. El onboarding de un nuevo equipo debe tomar menos de cuatro horas desde cero hasta el primer

deploy exitoso, con un golden path documentado paso a paso que no requiere conocimiento previo de la plataforma.

Los ciclos de feedback de la plataforma deben ser rápidos y cerrados. Cuando un equipo reporta un pain point en el canal de Slack de la plataforma, la expectativa debe ser una respuesta de reconocimiento en menos de 24 horas y una decisión sobre si se incluye en el backlog dentro de 72 horas. Cuando se lanza una nueva capacidad de la plataforma, el equipo de plataforma debe medir activamente si los equipos consumidores la usan, qué problemas encuentran en el primer contacto, y qué cambios la harían más adopta. Este ciclo de diseño centrado en el usuario no es diferente al de cualquier producto de software: la plataforma tiene usuarios, y la satisfacción de esos usuarios es el único indicador que importa.

Indicadores de una plataforma que multiplica productividad

- **Adoption rate superior al 80%** de los equipos de AI Engineering elegibles dentro de los primeros seis meses de lanzamiento de un nuevo feature; por debajo del 80%, el feature no resuelve un problema real o tiene problemas de usabilidad.
- **Time-to-first-deploy inferior a 4 horas** para un nuevo equipo que se integra a la plataforma por primera vez; este número captura la calidad del onboarding, la documentación y la experiencia inicial de usuario.
- **Reduction in infrastructure-related postmortems:** caída medible trimestre a trimestre en incidentes cuya causa raíz es configuración de infraestructura; indica que la plataforma está eliminando clases enteras de errores, no solo mitigándolos.

Nota del Arquitecto: Una plataforma de IA bien diseñada hace que hacer las cosas bien sea más fácil que hacerlas mal: los guardrails técnicos y los golden paths deben ser el camino de menor resistencia, no obstáculos. Cuando el cumplimiento de las políticas de seguridad, governance y calidad es automático e invisible para el equipo, la plataforma ha logrado su diseño más ambicioso: convertir las buenas prácticas en el comportamiento por defecto.

La Ley de Conway, que establece que las organizaciones diseñan sistemas que son una copia de sus estructuras de comunicación, opera también sobre las plataformas de IA: una plataforma diseñada por un equipo desconectado de sus usuarios reflejará las necesidades del equipo de plataforma, no las de los ML Engineers. El capítulo siguiente inicia el recorrido por

los componentes especializados de la plataforma, comenzando por el que más impacto directo tiene en la reproducibilidad y el governance: el model registry.

"Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure."

— Melvin Conway, ingeniero de software, formulador de la Ley de Conway (1967)